

$D(SP)^2$ – Status do Projeto

Biblioteca DSP, conversão áudio-MIDI e orquestra de motores de passo

Caio Davi de Andrade Monteiro, Rodrigo de Souza Oliveira, Luis
Rafael Sena

Engenharia Eletrônica e de Computação

May 5, 2026

Código geral da biblioteca

O núcleo do projeto já está estruturado como uma biblioteca DSP híbrida: core em C++ para desempenho e bindings em Python para orquestração.

Base implementada

- grafo dataflow com engine C++;
- roteamento por buffers;
- bindings Python via pybind11;
- build separado para simulação e embarcado.

Nós e ferramentas

- osciladores, filtros e matemática básica;
- FFT, espectro, picos e pitch;
- leitura WAV e exportação MIDI;
- relatórios, gráficos e logger.

Validação

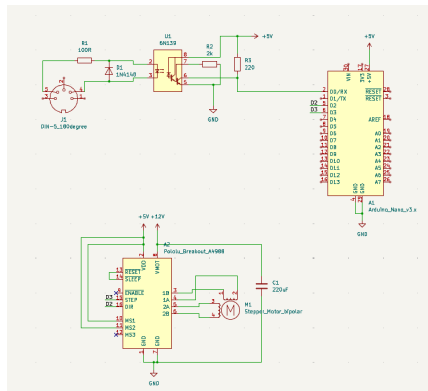
O repositório tem 10 arquivos de teste: 5 C++/CTest e 5 Python/unittest, somando 81 cenários nomeados.

Orquestra de passo e hardware

- circuito já projetado e simulado;
- componentes principais já foram comprados;
- ainda não houve teste físico do hardware;
- com a peça final, partimos para montagem e testes.

Ponto importante

Hardware e software avançam em paralelo: enquanto a placa pendente não chega, seguimos validando a biblioteca e o MIDI.



Componentes da orquestra

Comprados

- cabo de áudio MIDI USB;
- 6x driver de motor de passo A4988;
- 6x motor de passo NEMA 17 17HS4401;
- placa Nano V3 com pinos soldados.

Pendente

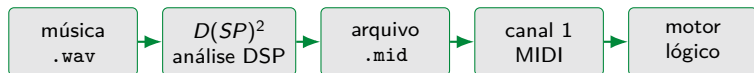
- placa adaptador mini shield para Arduino.

Estado atual

A montagem física começa assim que essa placa chegar.

MVP: música para MIDI

O MVP transforma uma música em um arquivo MIDI de um canal, que representa um motor lógico no fluxo de software.



- peaks: detecta picos espectrais por bloco.
- melody: estima a fundamental e gera uma linha melódica.
- saída atual: um canal MIDI, preparado para a primeira validação física.

Já temos exemplos .mid gerados no projeto, incluindo testes com *Twinkle Twinkle Little Star* e Bach.

Evidência: MIDI gerado

O que já funciona

- entrada de áudio WAV mono;
- análise espectral e detecção de notas;
- escrita de MIDI em um canal;
- API Python e linha de comando.



Resultado esperado

O arquivo MIDI é a interface entre a biblioteca e a futura montagem: primeiro validamos um canal/motor; depois expandimos para vários canais.

- o código Arduino já está pronto;
- ele recebe eventos MIDI;
- cada nota é convertida em período de pulso;
- o código já prevê o mapeamento por canal/motor.

Estado correto

Ainda não testamos o hardware. O Arduino está pronto para teste, mas a validação real depende da montagem do circuito.

Estratégia de validação

Primeiro testar um canal MIDI com um motor físico. Depois, ajustar temporização e expandir para múltiplos motores sincronizados.

MVP atual

- software gera .mid de um canal;
- esse canal representa um motor lógico;
- ainda não houve teste com motor físico.

Próxima expansão

- canal MIDI 1 → motor 1;
- canal MIDI 2 → motor 2;
- ...
- canal MIDI 6 → motor 6.

O objetivo depois da validação inicial é sincronizar vários canais para tocar acordes com uma voz por motor.

Próximos passos

Software

- refinar estabilidade das notas;
- melhorar detecção melódica;
- preparar MIDI multicanal.

Hardware

- receber o mini shield;
- montar o circuito;
- validar acionamento real.

Integração

- testar primeiro um canal;
- ajustar temporização;
- evoluir para acordes.

Resumo

A parte de software não precisa esperar o hardware terminar: quando o mini shield chegar, já teremos a cadeia áudio-MIDI e o Arduino prontos para os primeiros testes e ajustes na montagem real.